

CDC IP Creation & Clock Groups

Sequential runbook • July 2026

From sim-verified `rtl/cdc` sources to five packaged IPs, integrated across three clock domains, with a clean `report_cdc`.

You drive Vivado — every step below is a command you run or a box you tick. Each part ends with a **checkpoint**; do not proceed past a failed one. Companion docs: `docs/ip_packaging_runbook.md` (original GUI walkthrough) and `rtl/cdc/README_cdc.md` (XPM production-swap table).

Part	What you do	Vivado
0	Prerequisites, part choice, and a mandatory XDC fix	no
1	Package the five IPs by hand in the GUI	yes
2	Register the IP repository	yes
3	Verify each IP out-of-context (<code>report_cdc</code> per block)	yes
4	Define the three clock domains and their groups	yes
5	Integrate and verify the full design	yes
6	Trap reference	—
A	Module port/parameter reference for IP Integrator wiring	—

1. Prerequisites and mandatory fixes

Do all of Part 0 before opening Vivado. Section 0.3 in particular is a real defect: left alone it silently produces an unconstrained design that still looks green.

0.1 Confirm the library still passes simulation

```
cd sim && make cdc-regression
```

CHECKPOINT 0.1 — ends with `==== CDC regression PASSED =====`, 8 tests across all five blocks. If red, fix before packaging — do not bake a broken block into an IP.

What this proves — and does not. `cdc-regression` runs under Verilator, a *zero-delay cycle simulator*. It proves protocol and data integrity across unrelated clocks. It **cannot** model metastability or a setup/hold violation. Green here does *not* mean the crossing is silicon-safe — that is what Part 3 is for. Keep these two claims separate when describing this work.

0.2 Pick the right part — `xcvu47p` is not installed

You will select a part in the **New Project** wizard five times (once per block), so decide now. Use:

```
xcu200-fsgd2104-2-e
```

Do **not** pick the F2 part `xcvu47p-fsvh2892-2L-e-es1`. Your local Vivado 2025.2 install has **78 parts**, families `virtexuplus`, `zynq`, `azynq` only — `xcvu47p` is **not among them**. It exists only on the AWS FPGA Developer AMI build host, so it will not appear in the wizard's part list no matter how you filter.

`xcu200-fsgd2104-2-e` is the part every existing synth result in `results/` was produced against, so your numbers stay comparable. In the wizard: **Parts** tab, type `xcu200` in the search box, select the single match.

To confirm from the Tcl Console at any time:

```
llength [get_parts xcu200-fsgd2104-2-e]    ;# expect 1
llength [get_parts xcvu47p*]               ;# expect 0 locally
```

When you later package on the AWS host, choose the F2 part there instead. The CDC logic is device-agnostic; only the packaging target changes.

Aside: the scripted alternative `ip/package_cdc_ips.tcl` hard-codes `xcvu47p` on line 11 and therefore aborts at `create_project` on this machine. You are doing this manually, so it does not affect you — but if you ever fall back to the script, fix that line first.

0.3 Fix four XDC files whose constraints match zero cells

This one is silent, which makes it the dangerous one. Several constraints select cells with a filter like:

```
[get_cells -hierarchical -filter {NAME =~ *wq_reg[0]*}]
```

In Tcl's string match, [0] is a **character class**, not a literal bracket. It matches the single character 0, so the pattern expects wq_reg followed immediately by the character 0. The actual synthesized cell is named wq_reg[0][3] — the next character is [. **The pattern matches nothing.** Verified with plain Tcl:

```
*wq_reg[0]*      vs wq_reg[0][3]      -> NO MATCH
*wq_reg\[0\]*    vs wq_reg[0][3]      -> MATCH
*sync_reg[0]*    vs u_fwd/sync_reg[0]  -> NO MATCH
*sync_reg\[0\]*  vs u_fwd/sync_reg[0]  -> MATCH
*sync_reg\[0\]*  vs u_fwd/sync_reg[1]  -> NO MATCH    (correctly selective)
```

An empty object list means set_max_delay / set_false_path applies to **nothing**. Synthesis still succeeds, simulation stays green, report_cdc may still look reasonable — and your crossing is unconstrained. Escape the brackets.

The edits. In rtl/cdc/xdc/sync_2ff_ooc.xdc:

```
set_false_path -to [get_cells -hierarchical -filter {NAME =~ *sync_reg\[0\]*}]
```

In rtl/cdc/xdc/pulse_sync_ooc.xdc — both set_max_delay targets:

```
-to [get_cells -hierarchical -filter {NAME =~ *u_fwd*sync_reg\[0\]*}]
-to [get_cells -hierarchical -filter {NAME =~ *u_ack*sync_reg\[0\]*}]
```

In rtl/cdc/xdc/async_fifo_ooc.xdc and rtl/cdc/xdc/axis_async_fifo_ooc.xdc:

```
-to [get_cells -hierarchical -filter {NAME =~ *wq_reg\[0\]*}]
-to [get_cells -hierarchical -filter {NAME =~ *rq_reg\[0\]*}]
```

Not affected — these use no index and already match correctly: every set_property ASYNC_REG line, and handshake_mcp_ooc.xdc's *data_hold_reg* → *dst_data_reg* set_max_delay.

0.4 Decide your three domain frequencies

The periods baked into the OOC XDCs are *representative placeholders* for out-of-context synthesis. They are overridden by the real clocks at integration (Part 4), but pick your intended numbers now — they drive each IP's FREQ_HZ.

Domain	Role	Current OOC placeholder
Line / RX	market-data ingest	156.25 MHz (6.400 ns)
Core	parser → book → strategy → risk	250 MHz (4.000 ns)
Host / CSR	AXI-Lite, DMA, telemetry	125 MHz (8.000 ns)

Reality check on Core. Your closed result today is **200 MHz** (tick_to_trade_top with the strategy selector, WNS +1.815 ns). The XDCs say 250 MHz. Either retime for 250 or set the core domain to 200 MHz and update the periods — but do not leave a constraint claiming a frequency you have not closed.

CHECKPOINT 0 — regression green; PART fixed; four XDCs escaped; three frequencies chosen and written down.

2. Package the five IPs

Each block is self-contained (`pulse_sync` and `handshake_mcp` include their own copy of `sync_2ff.sv`; `axis_async_fifo` includes `async_fifo.sv`), so **there is no ordering dependency** — package them in any order.

You do this entirely by hand in the Vivado GUI — one project per block, adding the source files yourself. Repeat steps 1.1–1.7 five times, substituting the block name and its files from this table each pass:

Pass	Block (= top)	Design sources to add	Constraints to add
1	<code>sync_2ff</code>	<code>rtl/cdc/sync_2ff.sv</code>	<code>sync_2ff_ooc.xdc</code>
2	<code>pulse_sync</code>	<code>rtl/cdc/pulse_sync.sv</code> <code>rtl/cdc/sync_2ff.sv</code>	<code>pulse_sync_ooc.xdc</code>
3	<code>async_fifo</code>	<code>rtl/cdc/async_fifo.sv</code>	<code>async_fifo_ooc.xdc</code>
4	<code>axis_async_fifo</code>	<code>rtl/cdc/axis_async_fifo.sv</code> <code>rtl/cdc/async_fifo.sv</code>	<code>axis_async_fifo_ooc.xdc</code>
5	<code>handshake_mcp</code>	<code>rtl/cdc/handshake_mcp.sv</code> <code>rtl/cdc/sync_2ff.sv</code>	<code>handshake_mcp_ooc.xdc</code>

All constraint files live in `rtl/cdc/xdc/`. Blocks that instantiate a sub-module need *both* files — miss the second one and synthesis fails with **ERROR: [Synth 8-439] module '`<name>`' not found.**

Add the sub-module as a *file*, not as a packaged IP. When `pulse_sync` instantiates `sync_2ff`, Vivado needs the RTL text of `sync_2ff` in *pulse_sync's own source list*. It does not resolve an RTL module instantiation against your IP catalog, so the `sync_2ff` IP you packaged in pass 1 is irrelevant here — add `rtl/cdc/sync_2ff.sv` to the project like any other source.

That means `sync_2ff.sv` ends up inside three IPs and `async_fifo.sv` inside two. Deliberate: every IP is **self-contained**, so dropping any one of them into a design pulls in everything it needs, and the five passes have no ordering dependency at all.

1.1 Create the project

1. **File** → **Project** → **New...** → *Next*.
2. **Project name:** `pkg_<block>` (e.g. `pkg_sync_2ff`). **Location:** a `projects/` folder that is **not** inside the IP location you will choose in 1.5 — e.g. `<ip-root>/projects/`. These are scratch projects, not deliverables. Tick *Create project subdirectory*. → *Next*.
Nesting the project inside the IP folder is what produces 19-3898 (“file is not found in the project sources”) and 19-3656 (“repository path may become invalid”). Vivado wants the two adjacent, e.g.:

```
IPs/projects/pkg_pulse_sync/    <- the Vivado project
IPs/cdc_repo/pulse_sync/       <- the packaged IP (component.xml)
```

3. **Project type:** *RTL Project*. Tick *Do not specify sources at this time* — you will add them deliberately in 1.2. → *Next*.
4. **Default Part:** *Parts* tab, search `xcu200`, select `xcu200-fsgd2104-2-e` (see Part 0.2). → *Next* → *Finish*.

1.2 Add the design sources by hand

1. Flow Navigator → **PROJECT MANAGER** → **Add Sources** (or **Alt+A**).
2. Select *Add or create design sources* → *Next*.
3. **Add Files...** → browse to `rtl/cdc/` → select the `.sv` file(s) for this pass from the table above (Ctrl-click for two).
4. Tick **Copy sources into project**. **Do not skip this.** If left unticked the project references the files in place (`$PPRDIR/../../Projects/HFT/rtl/cdc/...`) while the packager makes its own copies under `<ip-root>/src/`. The two paths do not match, so on merge the packager reports 19-3898 and **drops the file from the packaged IP**. A project with no `.srcs/` directory is the tell that this was missed.
5. *Finish*.
6. In the **Sources** pane, confirm each file's type is **SystemVerilog**. If one came in as plain Verilog, right-click it → **Set File Type...** → *SystemVerilog*. These files use `logic`, `always_ff` and packed arrays — as Verilog-2001 they will throw syntax errors.

1.3 Set the top module

In **Sources** → **Design Sources**, right-click the block's module → **Set as Top**. The top module's name renders in **bold**.

If **Set as Top** does not appear, the module is already top — Vivado hides the option for the module that already holds it, and with only one module in the project it auto-designates that one. On pass 1 (`sync_2ff`) that is always the case, so there is nothing to do. Treat this step as a *verification*: it only bites on passes 2, 4 and 5, where you must confirm the *wrapper* is top and not the sub-module it instantiates. (Two other causes of a missing option: right-clicking in the **Libraries** or **Compile Order** tab instead of **Hierarchy**, or a file that failed to parse and landed under *Non-module Files*.)

1.4 Add the OOC constraints

1. **Add Sources** again → *Add or create constraints* → *Next*.
2. **Add Files...** → `rtl/cdc/xdc/<block>_ooc.xdc` → tick **Copy constraints files into project** → *Finish*.
3. Select the XDC under **Sources** → **Constraints**. In the **Source File Properties** pane find **USED_IN** and tick all three: **synthesis**, **implementation**, **out_of_context**.

Step 3 is not optional. Without `out_of_context` in **USED_IN**, the CDC exceptions in that XDC are ignored whenever the IP is synthesized standalone — and standalone is exactly how a packaged OOC IP gets built. The IP will look fine and be unconstrained.

1.5 Launch the packager

1. **Tools** → **Create and Package New IP...** → *Next*.
2. Choose **Package your current project** → *Next*.
3. **IP location:** `<ip-root>/cdc_repo/<block>` — a **per-block subfolder** (create it if the browser does not). Keep all five under the same `cdc_repo` parent: that single directory becomes your IP repository in Part 2. → *Next*.
4. *Finish*. The **Package IP** window opens with a left-hand step list.

Give every block its own folder. If you point the IP location at a bare directory (say `~/IPs`) rather than `~/IPs/cdc_repo/<block>`, Vivado writes `component.xml` to the *top* of it — and the next block you package overwrites it. You finish with one IP instead of five, and Part 2 has nothing to find. Check after each pass that `component.xml` sits in a per-block subfolder, not at the repository root.

1.6 Work through the Package IP tabs

Click each entry in the left panel in order. A tab showing a warning triangle needs attention; one showing a tick is done.

- **Identification** — Vendor `hft`, Library `cdc`, Name `<block>`, Version `1.0`, Display name `<block>`, Description “HFT CDC primitive: `<block>`”, Taxonomy `/HFT/CDC`.
- **File Groups** — *the manifest, and the tab that matters most*. It declares which files actually ship inside the IP and in what role: the **Synthesis** group is the RTL used when the IP is synthesized *plus your XDC*; **Simulation** is the RTL used in simulation. Confirm all three:
 - (a) the block's `.sv` appears under *Synthesis and Simulation*;
 - (b) the `_ooc.xdc` appears under *Synthesis* with file type XDC;
 - (c) on passes 2, 4 and 5, **both** `.sv` files are listed — wrapper plus sub-module.

If the tab offers **Merge changes from File Groups Wizard**, click it: the file list has gone stale, usually because you added the XDC in step 1.4 after opening the packager. That is nearly always why a missing XDC is missing.

Finally, check the **Is Include** column: it must be **unticked** on the XDC. `isIncludeFile` marks a file as a *header* pulled in via `'include` — meaningless for constraints, and it raises 19-4296.

- **Customization Parameters** — **Merge changes from Customization Parameters Wizard**, then verify the block's parameters are listed and editable: `sync_2ff` → `STAGES`, `INIT_VAL`; `pulse_sync` → `STAGES`; `async_fifo` → `WIDTH`, `DEPTH`, `STAGES`, `AF_LEVEL`, `AE_LEVEL`; `axis_async_fifo` → `TDATA_W`, `DEPTH`, `STAGES`; `handshake_mcp` → `WIDTH`, `STAGES`.

- **Ports and Interfaces** — the important tab. For each clock port, select it and set **FREQ_HZ** (your Part 0.4 numbers) and **ASSOCIATED_RESET**: `clk→rst_n`; `wr_clk→wr_rst_n`; `rd_clk→rd_rst_n`; `s_clk→s_rst_n`; `m_clk→m_rst_n`; `src_clk→src_rst_n`; `dst_clk→dst_rst_n`.

On `axis_async_fifo` only: confirm `s_axis_*` and `m_axis_*` have been inferred as **AXI4-Stream** slave/master interfaces. If they have not, right-click in the interface list → **Auto Infer Interface** → *AXI4-Stream*, or add the bus interface manually and map the ports by name. Then set **ASSOCIATED_BUSIF**: `s_clk→S_AXIS`, `m_clk→M_AXIS`. Without this the block design will not let you drag-connect the streams.

- **Review and Package** — read the summary, then click **Package IP**.

1.6a Ports and Interfaces in detail

This tab declares what your ports *mean* — which are clocks, which are resets, which belong to a bus — so IP Integrator can auto-connect them. Without it IPI sees anonymous wires.

How to set a value: right-click the *interface* (not the raw port) → **Edit Interface...** → the **Parameters** tab. If the parameter is not listed — a freshly inferred clock has *no* parameters at all — click **+** to add it by name, then set its value. **POLARITY** is a dropdown (it carries a choice list) rather than free text. Plain data ports — `d`, `q`, the **AXIS** payload signals — need nothing.

Equivalently, from the Tcl Console (faster, and easy to verify afterwards):

```
set core [ipx::current_core]
set clkif [ipx::get_bus_interfaces clk -of_objects $core]

ipx::add_bus_parameter FREQ_HZ $clkif
set_property value 250000000 [ipx::get_bus_parameters FREQ_HZ -of_objects $clkif]

ipx::add_bus_parameter ASSOCIATED_RESET $clkif
set_property value rst_n [ipx::get_bus_parameters ASSOCIATED_RESET -of_objects $clkif]

ipx::update_checksums $core
ipx::save_core $core
```

A parameter that already exists needs no `add_bus_parameter` — just `set_property value`.

Block	Clock	FREQ_HZ	ASSOCIATED_RESET	ASSOCIATED_BUSIF
sync_2ff	clk	250 000 000	rst_n	—
pulse_sync	src_clk	125 000 000	src_rst_n	—
	dst_clk	250 000 000	dst_rst_n	—
async_fifo	wr_clk	156 250 000	wr_rst_n	—
	rd_clk	250 000 000	rd_rst_n	—
axis_async_fifo	s_clk	156 250 000	s_rst_n	S_AXIS
	m_clk	250 000 000	m_rst_n	M_AXIS
handshake_mcp	src_clk	125 000 000	src_rst_n	—
	dst_clk	250 000 000	dst_rst_n	—

The frequencies are the periods from each block's OOC XDC, so the IP metadata and the constraints agree. If you settled on a 200 MHz core in Part 0.4, use 200000000 wherever the table says 250 MHz.

Set POLARITY = ACTIVE_LOW on every reset interface. All five blocks use `always_ff @(... or negedge rst_n)` — asynchronous, active low. If **POLARITY** is left **ACTIVE_HIGH**, IP Integrator wires a reset that is asserted when it should be released: the block either sits in reset forever or never resets at all. In practice Vivado *does* infer this correctly from the `_n` suffix — on `sync_2ff` the reset came through as `busType reset` with **POLARITY = ACTIVE_LOW** already set, so there was nothing to change. Verify rather than assume — the check is cheap: `grep -A2 POLARITY <ip>/component.xml` should show **ACTIVE_LOW**.

Pass 4 is the odd one. `axis_async_fifo` is the only block with a bus. Before setting **ASSOCIATED_BUSIF**, confirm `s_axis_*` and `m_axis_*` were inferred as **AXI4-Stream** slave/master interfaces; if not, right-click in the interface list → **Auto Infer Interface** → *AXI4-Stream*, or add the bus interface and map the ports by name. Without it you cannot drag-connect the streams in the block design. On the other four blocks **ASSOCIATED_BUSIF** stays empty — which is exactly why 19-5661 never clears there.

The GUI will not let you add ASSOCIATED_BUSIF. Its *Add Parameter* dialog offers only the parameters it considers standard for a clock interface, and ASSOCIATED_BUSIF is usually absent — reported as “ASSOCIATED_BUSIF does not exist”. The parameter is valid; only the dialog is limited. Add it from the Tcl Console instead:

```
ipx::add_bus_parameter ASSOCIATED_BUSIF $clkif then set_property value s_axis [ipx::get_bus_parameters ASSOCIATED_BUSIF -of_objects $clkif]
```

Use the interface names Vivado actually created — it derives them from the port prefix, so they come out lowercase `s_axis` / `m_axis`, not the `S_AXIS` convention. Either case is accepted downstream.

1.6b Warnings you will see — and which ones matter

The **Messages** pane fills with IP Packager warnings on every pass. Most are noise. Note that Vivado groups them by *wizard*, and the badge on a left-panel tab counts that wizard’s messages — which is not always the tab where you fix the problem. Triage:

Message	Means	Do
19-11770 Clock interface <code>clk</code> has no <code>FREQ_HZ</code> parameter	the clock carries no declared frequency	FIX — Ports and Interfaces → select the clock → set FREQ_HZ (and ASSOCIATED_RESET)
19-11888 IP description is not meaningful	Description still defaults to the module name. <i>Reported under the Compatibility Wizard, so the badge lands on the Compatibility tab</i>	FIX (cosmetic) — but on the Identification tab, not Compatibility: write a real description
19-5661 Bus Interface <code>clk</code> does not have any bus interfaces associated with it	ASSOCIATED_BUSIF is empty	IGNORE on <code>sync_2ff</code> , <code>pulse_sync</code> , <code>async_fifo</code> , <code>handshake_mcp</code> — they have no AXI interface. Only <code>axis_async_fifo</code> must set it
19-4296 Unsupported include file should not have <code>isIncludeFile</code> set	the <i>Is Include</i> box is ticked on the XDC; that flag marks a ‘include header, not constraints	FIX — File Groups → untick <i>Is Include</i> on the XDC, re-package
19-5101 Packaging a component with a SystemVerilog top file is not fully supported	the packager’s SV support at top level is partial	IGNORE here — these blocks expose only logic scalars and packed vectors, which map cleanly. Only if ports or parameters come out wrong in IPI, add a thin Verilog-2001 wrapper for packaging
19-3656 repository path may become invalid if the project moves	the packaging project is nested <i>inside</i> the IP folder	IGNORE if it already works; for later passes put the project <i>adjacent</i> to the IP (Part 1.1)
19-2187 The Product Guide file is missing	a documentation-completeness check — commercial IPs ship a datasheet PDF the way Xilinx IPs ship their PG documents	IGNORE — nothing to attach on a hand-rolled block; appears on all five passes

Do not expect a clean sheet. 19-5661 and 19-2187 never clear — they persist on all five passes and that is correct behaviour, not a defect. They are *warnings*, not errors: **Package IP** stays clickable throughout. Only 19-11770 (`FREQ_HZ`) and 19-11888 (description) call for action. Note too that these are *metadata* problems — rearranging folders or re-creating the project will not touch them; you clear them by filling fields in the wizard.

Check the Identification defaults. Out of the box the component is named something like `Twinwinglabs:user:sync_2ff:1.0` — your Vivado install’s default vendor — or `user.org:user:sync_2ff:1.0` on a stock install. Set **Vendor** `hft`, **Library** `cdc` and **Taxonomy** `/HFT/CDC` on every pass, or the five blocks will not group together under *HFT/CDC* in the catalog in Part 2. To check what you actually shipped, `grep spirit:vendor <ip>/component.xml`, and confirm `FREQ_HZ` appears at all — zero hits means 1.6’s clock settings never took.

On the Compatibility tab: leave *Package for Vitis* unchecked — that is for Vitis acceleration kernels with an `ap_ctrl_hs` control protocol, not what these are. Do **not** tick *Ignore Freq_Hz*; you want `FREQ_HZ`. Confirm the **Family** list includes `virtexuplus`: `xcvu47p` is a Virtex UltraScale+ device, so even though that part is not installed locally the family is covered and these IPs stay usable when you later build on the AWS host.

1.7 Close and repeat

Vivado offers to close the project after packaging — accept. Return to 1.1 for the next block. Because each block carries its own copy of any sub-module, **there is no ordering dependency**: package them in whatever order you like.

Note on OOC synthesis: the Out-Of-Context choice is **not** made in the packaging wizard. It is chosen later, when you instantiate the IP and run **Generate Output Products** — set **Synthesis Options** to *Out of context per IP* so each IP synthesizes and caches independently.

CHECKPOINT 1 — five packaged IPs exist, one folder each, every one containing a `component.xml` — *and each one passes the checker*:

```
python3 ip/check_packaged_ip.py <ip-root>/*
```

It reads the IP-XACT only (it never launches Vivado) and exits non-zero if any **FAIL** remains. Trust it over the wizard's badges: it verifies what actually got written to disk.

Do not skip the checker. The GUI reports success while leaving real problems behind — observed on the first live pass, *after* the packager said it was done: `isIncludeFile` had migrated from the XDC onto *both* copies of `sync_2ff.sv` (so the module would be treated as an include header rather than compiled), and the clock interface had picked up a stray `POLARITY` while still missing `FREQ_HZ` and `ASSOCIATED_RESET`. None of that is visible without reading `component.xml`.

When you untick *Is Include*, untick it everywhere. The flag is per-file and per-file-group: the same `.sv` appears in both the Synthesis and Simulation groups and each carries its own checkbox. Clearing it on one row — or clearing it on the XDC and accidentally setting it on the sources — leaves you exactly where you started. Re-run the checker after every re-package.

3. Register the IP repository

In the project where you will assemble the design:

```
set_property ip_repo_paths [file normalize ip/cdc_repo] [current_project]
update_ip_catalog
```

You need a project open first. An IP repository is a project setting, so there is no such menu on the Welcome screen. Since Part 4 needs an integration project anyway, create it now: **File** → **Project** → **New...**, name `cdc_integration`, RTL Project, *Do not specify sources at this time*, part `xcu200-fsgd2104-2-e`. The repository registration then carries straight through to the block design.

GUI route (use this one): Flow Navigator → **PROJECT MANAGER** → **Settings** → **IP** → **Repository** → the + button → browse to your IP root → *Select*. Vivado reports how many IPs it found — **it should say 5**. Click *OK*, then **Refresh IP Catalog** if prompted. (Equivalent: **Window** → **IP Catalog**, right-click → **Add Repository...**)

Do not use *Manage IP*. It looks like the right feature and is not: Manage IP is a flow for generating IP customisations out-of-context, and it demands a directory containing a `manage_ip_project`. Pointing it at your IP root gives “The directory you selected is not a valid Manage IP location”. Adding a *repository* is the Settings route above.

If it finds fewer than five, one of the packaging passes did not complete; check which `ip/cdc_repo/<block>/` folder is missing its `component.xml` and redo that pass.

CHECKPOINT 2 — all five blocks appear in **Window** → **IP Catalog** under **HFT / CDC**.

4. Verify each IP out-of-context

Do this before integrating. Per-block OOC checks prove the primitives are sound; integration checks (Part 5) prove your *wiring* is sound. Debugging both at once is much harder. This is the verification layer that has never been run on this library.

3.1 Run the checks — GUI route

Synthesis only — do not run implementation. `report_cdc`, `report_clock_interaction` and `check_timing` all work on the *synthesized* netlist. CDC is a structural property of how signals cross domains, fully determined after synthesis; place & route adds nothing to the analysis. Packaging itself (Part 1) needs neither — the packager reads the RTL and metadata directly.

And ignore per-block area and power. `sync_2ff` is two flip-flops; its utilisation is a couple of registers and its power is rounding error. A synthesis-only power report is vectorless and low-confidence anyway (real numbers need post-route plus a SAIF). The metrics that mean something come from the *integrated* design in Part 5, measured against the existing baseline — WNS +1.815 ns / 7570 LUTs at 200 MHz for the selector top, and WNS +0.066 ns from the real `xcvu47p` P&R. The question worth answering is what the CDC restructuring costs against those, not what a synchroniser weighs alone.

Reuse the `pkg_<block>` projects you built in Part 1. They already contain the right sources, the OOC XDC and the correct part, so there is nothing to set up. For each block:

1. **File** → **Project** → **Open Recent** → `pkg_<block>`.
2. Flow Navigator → **SYNTHESIS** → **Run Synthesis**. Accept the defaults; these blocks synthesize in seconds.
3. When it finishes choose **Open Synthesized Design**.
4. **Reports** → **Timing** → **Report CDC...** → tick *Details* → *OK*. Results open in a docked pane; use *Export* to save to `results/cdc/<block>.cdc.rpt`.
5. **Reports** → **Timing** → **Report Clock Interaction...** → *OK*.
6. **Reports** → **Timing** → **Check Timing...** → *OK*.

Equivalent Tcl, if you prefer typing into the **Tcl Console** at the bottom of the same GUI window (this is where the 3.2 checks must go anyway):

```
report_cdc -details -file results/cdc/async_fifo_cdc.rpt
report_clock_interaction -file results/cdc/async_fifo_clkint.rpt
check_timing -verbose -file results/cdc/async_fifo_checktiming.rpt
```

For reference, the sources each block needs (top module = the first file) — the same set you added by hand in Part 1.2:

Block	Sources
<code>sync_2ff</code>	<code>sync_2ff.sv</code>
<code>pulse_sync</code>	<code>pulse_sync.sv</code> <code>sync_2ff.sv</code>
<code>async_fifo</code>	<code>async_fifo.sv</code>
<code>axis_async_fifo</code>	<code>axis_async_fifo.sv</code> <code>async_fifo.sv</code>
<code>handshake_mcp</code>	<code>handshake_mcp.sv</code> <code>sync_2ff.sv</code>

3.2 Confirm the constraints actually bound to cells

This is the Part 0.3 bug class. **After `synth_design`, before trusting any report:**

```
# how many flops actually carry ASYNC_REG?
llength [get_cells -hier -filter {ASYNC_REG}]

# do the escaped patterns now select real cells? (expect non-zero)
llength [get_cells -hierarchical -filter {NAME =~ *wq_reg\[0\]*}]
llength [get_cells -hierarchical -filter {NAME =~ *rq_reg\[0\]*}]

# and confirm the exceptions landed
llength [get_timing_exceptions]
```

Expected non-zero `ASYNC_REG` counts: `sync_2ff` = STAGES (2); `pulse_sync` and `handshake_mcp` = 2 chains × STAGES (4); `async_fifo` / `axis_async_fifo` = 2 × STAGES × (AW+1) bits.

If any count is 0, stop. The constraint is not applied and every report below is meaningless.

3.3 Acceptance criteria

report_cdc — every crossing recognized and **Safe**; zero Unsafe, zero Unknown.

Block	Expect
sync_2ff	d is an async input into a 2-FF ASYNC_REG chain, false-pathed. Single clock, so no clock-pair crossing.
pulse_sync	Both toggle chains (u_fwd, u_ack) safe, 1-bit each.
async_fifo	Gray-coded pointer crossings classified safe (multi-bit but Gray, one bit changes per step).
axis_async_fifo	Same as async_fifo — a thin {tlast,tdata} wrapper.
handshake_mcp	1-bit req/ack toggles safe; the wide data_hold → dst_data bus recognized as an MCP crossing covered by set_max_delay -datapath_only .

Read **handshake_mcp** and the FIFOs carefully rather than just checking for green. If the wide bus is reported as an **unsafe multi-bit crossing**, the **set_max_delay** is not matching cell names (Part 0.3 again). The FIFO memory may also be flagged as a multi-bit crossing — that one is expected and justified (reads are gated by the synchronized pointer), and it is exactly the case you should be able to *explain* rather than waive.

report_clock_interaction — every async clock pair must read “**Asynchronous Groups**”. If a pair reads “**Partial False Path**”, that is the classic silent bug: some paths across the boundary are constrained and some are not. Treat it as a failure, not a warning.

check_timing — no unconstrained endpoints, no missing **create_clock**, no **no_clock** warnings.

CHECKPOINT 3 — for all five blocks — non-zero ASYNC_REG counts, zero Unsafe/Unknown in **report_cdc**, no “Partial False Path”, clean **check_timing**. Save the reports to **results/cdc/**: they are the evidence that the library is sound.

5. Clock domains and clock groups

4.1 The three domains and where each crossing sits

LINE domain (RX / ingest)	CORE domain (parse → book → strategy → risk)	HOST domain (AXI-Lite / DMA)
ITCH bytes --[axis_async_fifo]-->	itch_parser ... risk_check	
s_clk=line		
m_clk=core		+--[axis_async_fifo]--> egress/DMA
		s_clk=core, m_clk=host
config (strat_sel, thresholds) <--[handshake_mcp]--	host CSR writes	
	src=host, dst=core	
halt / enable bit	<--[sync_2ff]-----	host
GO / latency-clear strobe	<--[pulse_sync]-----	host
latency snapshots	--[handshake_mcp]-->	host telemetry
	src=core, dst=host	

Rule: no signal crosses a boundary except through one of these five blocks. A signal that straddles the boundary in raw logic is what turns **report_cdc** *Unknown*.

4.2 Write the top-level clock constraints

At integration the IP-level OOC **create_clock** commands are replaced by the real top-level clocks (shell clock recipe, an MMCM, or board pins). Define all three, then declare all three mutually asynchronous:

```
# --- top-level clock definitions (adjust to your actual sources) -----
create_clock -name clk_line -period 6.400 [get_ports clk_line]    ;# 156.25 MHz
create_clock -name clk_core -period 4.000 [get_ports clk_core]    ;# 250 MHz (see 0.4)
create_clock -name clk_host -period 8.000 [get_ports clk_host]    ;# 125 MHz

# --- the three domains are mutually unrelated -----
set_clock_groups -asynchronous \
```

```
-group [get_clocks clk_line] \
-group [get_clocks clk_core] \
-group [get_clocks clk_host]
```

One `set_clock_groups` with three `-group` options declares all three pairs async — do not write three separate two-group commands, which is easy to get subtly wrong. If the clocks come from an MMCM rather than separate oscillators, use the generated clock names (`get_clocks -of_objects [get_pins ...]`) instead of port names.

If you later move to an Arty A7-100T: that board has a *single* 100 MHz oscillator, so MMCM-derived clocks are *phase-related* and Vivado sees them as synchronous. You can still constrain them async and exercise the logic, but it is only a partial async-CDC demonstration — two truly independent oscillators are what makes this real. Also, Artix-7 device support is not installed in your Vivado today (`get_parts xc7a100t*` returns 0); you would add it via the installer.

4.3 Assemble in IP Integrator

Create the ports first, before adding any IP — right-click canvas → **Create Port...** (Ctrl+K). Setting Type=Clock with a frequency is what makes IPI propagate it into each IP's `FREQ_HZ`.

Port	Direction	Type	Setting
<code>clk_line</code>	Input	Clock	156.25 MHz
<code>clk_core</code>	Input	Clock	250 MHz
<code>clk_host</code>	Input	Clock	125 MHz
<code>rst_line_n</code>	Input	Reset	ACTIVE_LOW
<code>rst_core_n</code>	Input	Reset	ACTIVE_LOW
<code>rst_host_n</code>	Input	Reset	ACTIVE_LOW

Then **Add IP** (right-click canvas): `axis_async_fifo` ×2 (ingress + egress), `handshake_mcp`, `sync_2ff`, `pulse_sync`. Double-click each to set parameters — ingress `TDATA_W=8`, egress `TDATA_W=9`, `handshake_mcp` `WIDTH=32`, `STAGES=2` throughout.

To draw a connection: hover a pin until the cursor becomes a pencil, then drag to the target; valid targets highlight green. Work *from each port outward* rather than block by block — far fewer canvas jumps. Eighteen connections in total:

From port	To pins
<code>clk_line</code>	<code>axis0/s_clk</code>
<code>clk_core</code>	<code>axis0/m_clk</code> , <code>axis1/s_clk</code> , <code>handshake/dst_clk</code> , <code>sync_2ff/clk</code> , <code>pulse_sync/dst_clk</code>
<code>clk_host</code>	<code>axis1/m_clk</code> , <code>handshake/src_clk</code> , <code>pulse_sync/src_clk</code>
<code>rst_line_n</code>	<code>axis0/s_rst_n</code>
<code>rst_core_n</code>	<code>axis0/m_rst_n</code> , <code>axis1/s_rst_n</code> , <code>handshake/dst_rst_n</code> , <code>sync_2ff/rst_n</code> , <code>pulse_sync/dst_rst_n</code>
<code>rst_host_n</code>	<code>axis1/m_rst_n</code> , <code>handshake/src_rst_n</code> , <code>pulse_sync/src_rst_n</code>

Each reset follows its clock exactly. `clk_core` and `rst_core_n` go to the same five pins; likewise for the other two domains. A reset landing on a pin whose clock came from a different domain is the mistake to look for — it is silent in the block design and only shows up as odd behaviour later. Never fan one reset across all three domains.

Finally, export the data ports: select each remaining pin → **Ctrl+T** (Make External). For `AXIS`, select the *interface* pin (the bundled connector) and one `Ctrl+T` exports the whole bus: `S_AXIS` and `M_AXIS` on both FIFOs. Then individually: `handshake_mcp`'s `src_valid/src_data/src_ready/dst_valid/dst_data`; `sync_2ff`'s `d/q`; `pulse_sync`'s `src_pulse/src_busy/dst_pulse`. Nothing may be left dangling — `Validate` fails on unconnected pins and names the offender.

CHECKPOINT 4 — block design validates (**Tools** → **Validate Design**) with no critical warnings; every cross-domain connection passes through a CDC IP.

6. Verify the integrated design

After synthesizing the assembled design:

```
report_cdc                -details -file results/cdc/top_cdc.rpt
report_clock_interaction -file results/cdc/top_clkint.rpt
report_timing_summary    -file results/cdc/top_timing.rpt
check_timing             -verbose -file results/cdc/top_checktiming.rpt
```

Acceptance gates:

1. **report_cdc**: zero Unsafe, zero Unknown. An *Unknown* almost always means a signal bypassed the CDC IPs — fix it by routing through the right primitive, **not** by adding a waiver.
2. **report_clock_interaction**: all three pairs show “Asynchronous Groups”; no “Partial False Path”.
3. **report_timing_summary**: each clock meets its own period independently; WNS positive per domain.
4. **check_timing**: no unconstrained endpoints.

CHECKPOINT 5 — all four gates pass. At that point you have a genuine three-domain design with *static CDC sign-off* — a materially stronger claim than “the simulation passed”.

7. Trap reference

#	Trap	Symptom	Fix
1	Reaching for the F2 part xcvu47p	it is not in the New Project part list at all	Part 0.2 — use xcu200-fsgd2104-2-e
2	[0] in a NAME =~ filter is a character class	constraint silently applies to nothing	Part 0.3 — escape as \[0\]
3	XDC missing USED_IN out_of_context	exceptions ignored during OOC synth	set the file property in Part 1.3
4	set_property ASYNC_REG matched zero cells	synchronizers unprotected, sim still green	count with get_cells -hier -filter {ASYNC_REG} (3.2)
5	“Partial False Path” in clock interaction	some boundary paths unconstrained	one set_clock_groups covering all pairs (4.2)
6	Constraint claims 250 MHz, design closes 200	timing signed off against fiction	Part 0.4 — set the real number
7	Treating Verilator green as CDC sign-off	metastability never modelled	Part 3 is the real check
8	One reset fanned across all domains	reset removal violates in two domains	per-domain synchronized reset (4.3)
9	Only the wrapper .sv added on passes 2, 4, 5	elaboration fails, sub-module not found	Part 1.2 — add both files
10	.sv added as Verilog-2001, not SystemVerilog	syntax errors on logic / always_ff	Part 1.2 — Set File Type → SystemVerilog
11	Copy sources into project left unticked	19-3898: file “not found in the project sources”, XDC silently dropped from the IP	Part 1.2 — tick it; no .srcs/ dir is the tell
12	IP location is a bare folder, not a per-block subfolder	each pass overwrites the last component.xml; one IP instead of five	Part 1.5 — use cdc_repo/<block>
13	Spaces in the project path (e.g. IP for HFT)	Vivado copes, but Tcl paths, simulator scripts and shell tooling break later	Part 1.1 — use a space-free path
14	One project for all five blocks	Package your current project packages one IP from one top module	Part 1.1 — one project per block

Appendix A — Module reference for IPI wiring

sync_2ff — single-bit level crossing

Params STAGES=2, INIT_VAL=0.

Ports clk, rst_n, d (async in), q (synchronized out).

pulse_sync — one-cycle strobe crossing (toggle + closed-loop ack)

Params STAGES=2.

Src src_clk, src_rst_n, src_pulse, src_busy.

Dst dst_clk, dst_rst_n, dst_pulse.

src_busy is high while a pulse is in flight — gate new pulses on it or they are dropped.

async_fifo — multi-bit stream, Gray-pointer dual-clock, FWFT

Params WIDTH=8, DEPTH=16 (power of two), STAGES=2, AF_LEVEL=DEPTH-2, AE_LEVEL=1.

Write wr_clk, wr_rst_n, wr_en, wr_data, full, almost_full.

Read rd_clk, rd_rst_n, rd_en, rd_data, empty, almost_empty.

axis_async_fifo — AXI-Stream boundary crossing

Params TDATA_W=8, DEPTH=16, STAGES=2.

Slave s_clk, s_rst_n, s_axis_tvalid, s_axis_tready, s_axis_tdata, s_axis_tlast.

Master m_clk, m_rst_n, m_axis_tvalid, m_axis_tready, m_axis_tdata, m_axis_tlast.

Carries {tlast, tdata} through one async_fifo of width TDATA_W+1.

handshake_mcp — atomic multi-bit config (req/ack, data held stable)

Params WIDTH=32, STAGES=2.

Src src_clk, src_rst_n, src_valid, src_data, src_ready.

Dst dst_clk, dst_rst_n, dst_valid (1-cycle pulse), dst_data (held stable).

Load only when src_ready is high. Only the toggles cross; the data bus is an MCP relaxed with set_max_delay -datapath_only, never a synchronized bus.